

Task 1: Learning ROS2, TurtleBot3, SLAM and Nav2

Surendar

August 28, 2026

1 Introduction

This report summarizes what I learned while completing Task 1: setting up ROS2 Humble with Gazebo, understanding TurtleBot3, learning the basics of SLAM and Nav2, and finally running a simulated robot that builds a map and navigates to a goal point.

2 Environment Setup

I installed ROS2 Humble on Ubuntu 22.04 along with Gazebo Classic, which is the simulator compatible with this ROS2 distribution. I also set up a TurtleBot3 workspace (`turtlebot3_ws`) with the TurtleBot3 packages built from source, and set the `TURTLEBOT3_MODEL` environment variable (e.g. `burger` or `waffle_pi`) so the correct robot model gets spawned in simulation.

3 Learning TurtleBot3

TurtleBot3 is a small, ROS2-compatible differential-drive robot commonly used for learning navigation and SLAM. It comes in a few variants (Burger, Waffle, Waffle Pi) that differ in sensors and physical size. Key things I learned:

- The robot publishes odometry (`/odom`) and laser scan data (`/scan`) from its onboard LIDAR, and subscribes to `/cmd_vel` for velocity commands.
- `teleop_twist_keyboard` can be used to manually drive the robot using the keyboard, which is useful for testing before automating navigation.
- The robot model, sensors, and physical parameters (like wheel separation and diameter) are defined in an SDF/URDF file that gets loaded into Gazebo when the entity is spawned.

4 SLAM: Localization and Mapping

SLAM (Simultaneous Localization and Mapping) lets a robot build a map of an unknown environment while simultaneously tracking its own position within that map. I used the `slam_toolbox` package in its online asynchronous mode (`online_async.launch.py`), which:

- Subscribes to LIDAR scan data and odometry to incrementally build an occupancy grid map.
- Uses a solver plugin (Ceres Solver with the SCHUR-JACOBI preconditioner in this case) to optimize the pose graph as new scans come in.
- Publishes the map that can be visualized live in RViz.

Localization is the robot figuring out where it is on a map, while *mapping* is building that map in the first place – SLAM does both at once, which is why it is harder than doing either individually.

5 Frames and the TF Tree

ROS2 keeps track of the relationships between coordinate frames (e.g. `map`, `odom`, `base_link`, `base_scan`) using the `tf2` library. The command that visualizes the entire flow of frames and how they are connected is:

```
ros2 run tf2_tools view_frames
```

This generates a PDF showing the full TF tree, which is useful for debugging when a frame is missing or a transform is not being published correctly. For visualizing the node and topic connections (rather than frames), `rqt_graph` is the equivalent tool.

6 Nav2: Navigation

Nav2 is the ROS2 navigation stack that takes the map produced by SLAM and plans a path for the robot to reach a goal pose while avoiding obstacles. It brings together several components: a global planner (computes a path across the whole map), a local planner / controller (follows that path while avoiding obstacles in real time using the costmap), and behavior trees that coordinate recovery actions if the robot gets stuck. Once `nav2_bringup` is launched alongside the map, a goal pose can be sent through RViz using the “2D Goal Pose” tool, and the robot autonomously drives itself there.

7 Implementation

I chose the default TurtleBot3 world in Gazebo as my environment. The robot was spawned, driven manually to build up the map using `slam_toolbox`, and then Nav2 was used to send it to a goal point autonomously.

7.1 Spawning the Robot and Starting SLAM

Figure 1 shows the terminals used to spawn the TurtleBot3 (Burger model) into Gazebo, launch `gzclient` for visualization, start the `slam_toolbox` online async node, and run `teleop_twist_keyboard` to drive the robot around manually.

7.2 Building the Map

Figure 2 shows RViz and Gazebo side by side while the robot was driven around the environment. The occupancy grid map (grey/white/black) is being built live in RViz as the LIDAR scans (red points) are processed by `slam_toolbox`.

7.3 Navigating with Nav2

[Placeholder – to be replaced with a screenshot of Nav2 running: the completed map, the robot’s costmaps, and a goal pose being sent and followed in RViz.]

8 Single Launch File

To tie the whole task together, I wrote a single ROS2 launch file (`turtlebot3_slam_nav2_launch.py`) that starts Gazebo with the TurtleBot3 world, spawns the robot, starts `slam_toolbox`, brings up Nav2, and opens RViz – all with one command:

```
ros2 launch turtlebot3_slam_nav2_launch.py
```

The file and this report are both included in the submitted GitHub repository.

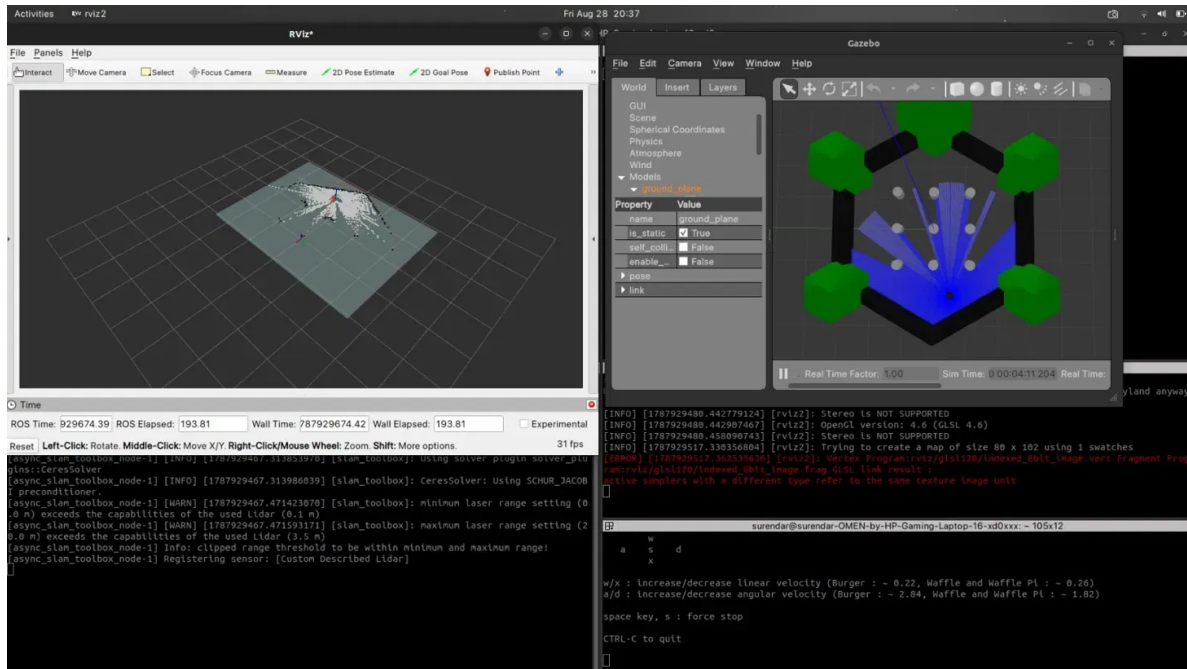


Figure 2: Live SLAM map being built in RViz alongside the Gazebo simulation.



Figure 3: Placeholder: Nav2 sending the robot to a goal pose on the saved map.